

Ejercicio 1 - Más capas en el perceptrón multicapa (Iris)

Notebooks

notebook 04:  04 Multilayer perceptron.ipynb

notebook 05:  05 Keras - multilayer perceptron - iris.ipynb

Análisis

Bajó más el error al añadir dos capas, o se estancó/empeoró? ¿Igual en NumPy y en Keras?

En ambas implementaciones la red más profunda terminó peor, pues el error/pérdida final subió en los dos casos. En NumPy el error se disparó, de 0.0553 a 0.6708, mientras que en Keras solo aumentó de 0.1343 a 0.2223. La curva de NumPy, además, se aplanó por completo, mientras que la de Keras no muestra un estancamiento tan extremo.

¿Las curvas de la notebook 01 (NumPy) y de Keras se parecen con la misma topología? Si no, ¿qué diferencias de implementación podrían explicarlo?

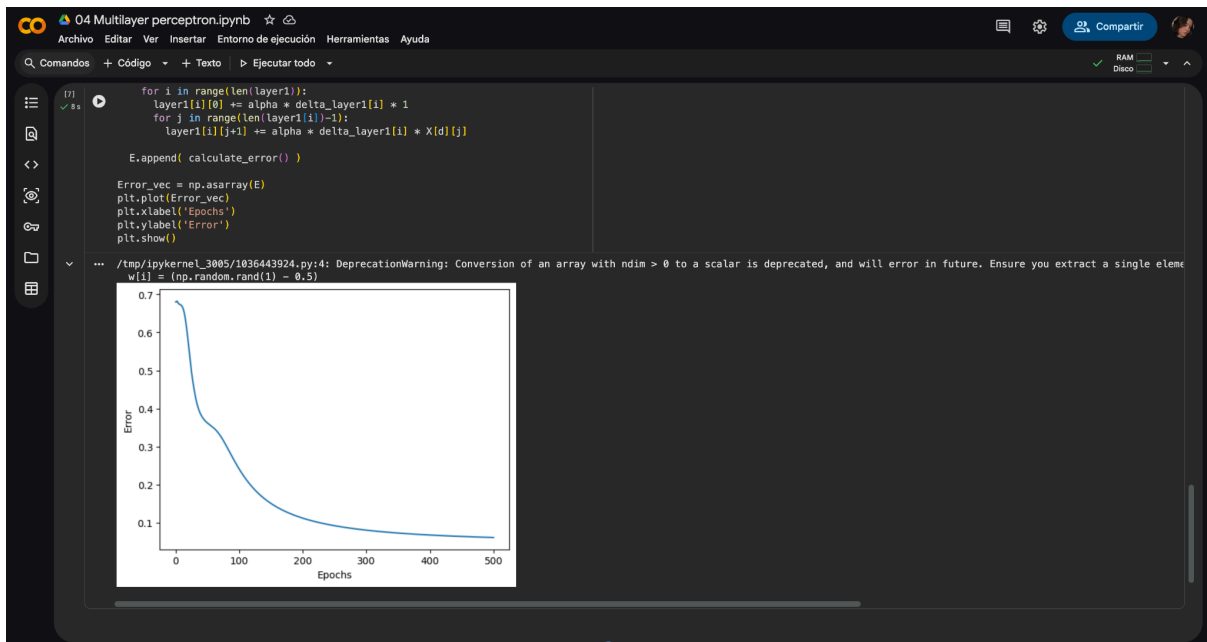
No son iguales. Por un lado, el orden de actualización de pesos es distinto: la notebook de NumPy actualiza los pesos ejemplo por ejemplo dentro de cada época (SGD puro), mientras que *model.fit* de Keras usa mini-batches, promediando el gradiente sobre cada lote antes de actualizar. De igual manera, NumPy usa valores uniformes en (-0.5, 0.5) para inicializar los pesos, mientras que Keras usa su inicializador por defecto (Glorot/Xavier uniform). También influye la escala del error: NumPy suma el error cuadrado de las 3 salidas sin dividir entre 3 (solo entre el número de ejemplos), mientras que el MSE de Keras promedia sobre todos los elementos (muestras $\times$ salidas). Por eso los números de NumPy y Keras no son directamente comparables en magnitud absoluta, pero sí lo es la comparación dentro de cada implementación (original vs. profunda).

Con sigmoides apiladas y MSE, ¿tiene sentido que una red más profunda no aprenda mejor en Iris?

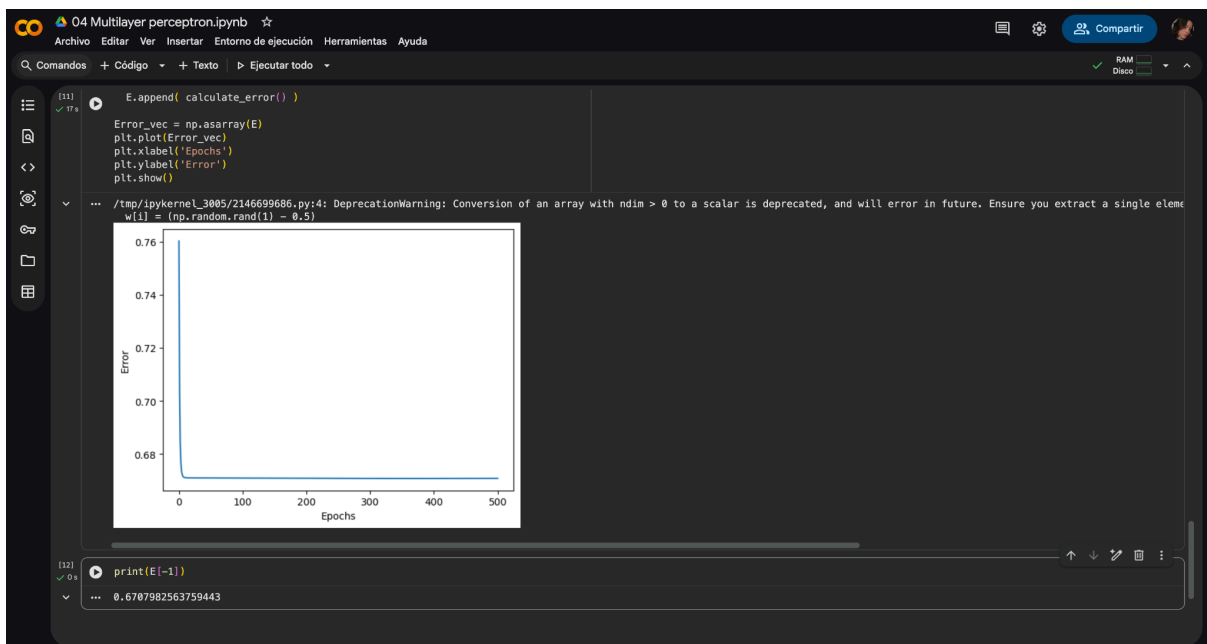
Sí, ya que Iris es un problema pequeño y casi linealmente separable por pares de clases, así que no necesita más capacidad de la que ya da una sola capa oculta de 3 neuronas. Al apilar sigmoides, cada capa extra multiplica el gradiente por un factor $h(1-h)$ adicional, así que el ajuste que le llega a las primeras capas se vuelve cada vez más pequeño. La curva aplanada de NumPy profunda confirma esto: la red se queda en un óptimo muy pobre casi desde el principio, porque las capas más profundas casi no reciben señal de error utilizable.

Evidencias

Gráfica de error — notebook 04, original (4×3×3)



Gráfica de error — notebook 04, profunda (4×3×3×3×3)



model.summary() y gráfica de loss — notebook 05, original (4×3×3)

05 Keras - multilayer perceptron - iris.ipynb

```
[3]: # 0 => [1 0 0]
# 1 => [0 1 0]
# 2 => [0 0 1]

[3]: idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])

[5.1 3.5 1.4 0.2]
0
[1 0 0]

[4]: model = keras.Sequential(
    [
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
        layers.Dense(3, activation="sigmoid", name="layer2"),
    ]
)

model.summary()

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models,
Model: "sequential"



| Layer (type)   | Output Shape | Param # |
|----------------|--------------|---------|
| layer1 (Dense) | (None, 3)    | 15      |
| layer2 (Dense) | (None, 3)    | 12      |



Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

[5]: y = model(X)
print(y[149])

tf.Tensor([0.4784917 0.47603348 0.39658207], shape=(3,), dtype=float32)
```

05 Keras - multilayer perceptron - iris.ipynb

```
[8]: pd.DataFrame(history.history).plot(figsize=(8,5))
plt.show()
```



```
[9]: model.predict(np.array([[3,3,1,1]]))

1/1 — 0s 107ms/step
array([[0.64232665, 0.28397618, 0.169328 ]], dtype=float32)
```

Comienza a programar o generar con IA.

Variables Terminal 2:29 p.m. Python 3

model.summary() (4 capas Dense) y gráfica de loss — notebook 05, profunda (4×3×3×3×3)

05 Keras - multilayer perceptron - iris.ipynb

ComandosCódigoTextoEjecutar todo

layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

```
[18]: model_deep = keras.Sequential([
      layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
      layers.Dense(3, activation="sigmoid", name="layer2"),
      layers.Dense(3, activation="sigmoid", name="layer3"),
      layers.Dense(3, activation="sigmoid", name="layer4"),
    ])

      model_deep.summary()
```

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

```
[19]: y = model(X)
      print(y[149])
```

05 Keras - multilayer perceptron - iris.ipynb

ComandosCódigoTextoEjecutar todo

5/5 Epoch 494/500 0s 8ms/step - loss: 0.2223

5/5 Epoch 495/500 0s 8ms/step - loss: 0.2223

5/5 Epoch 496/500 0s 9ms/step - loss: 0.2223

5/5 Epoch 497/500 0s 8ms/step - loss: 0.2223

5/5 Epoch 498/500 0s 8ms/step - loss: 0.2223

5/5 Epoch 499/500 0s 9ms/step - loss: 0.2223

5/5 Epoch 500/500 0s 9ms/step - loss: 0.2223

